CollabBoard Pre-Search Document

Project: Real-Time Collaborative Whiteboard with AI Agent **Date:** February 16, 2026 **Timeline:** 7-day sprint (MVP in 24 hours)

Executive Summary

This document outlines the architectural decisions, technology stack choices, and tradeoffs for building a production-scale collaborative whiteboard application with AI integration. The project emphasizes AI-first development workflows and real-time multiplayer synchronization.

Final Stack Decision:

- Frontend: React + TypeScript + Vite + Konva.js
- Backend: Firebase (Realtime Database + Firestore hybrid)
- AI: Anthropic Claude via Firebase Cloud Functions
- Auth: Firebase Authentication
- Deployment: Vercel (frontend) + Firebase (backend)

Phase 1: Constraints Definition

1. Scale & Load Profile

Launch Target: 5-10 concurrent users for demo/testing **6-Month Target:** 100-1,000 users **Traffic Pattern:** Spiky (workshop/meeting times during business hours) **Real-Time Requirements:** Critical - WebSocket/real-time sync essential **Cold Start Tolerance:** Low for board sync (<100ms), acceptable for AI commands (1-2s)

Rationale: Project is optimized for rapid MVP delivery with ability to scale to moderate production usage within free/low-cost tiers.

2. Budget & Cost Ceiling

Development Budget: $0-50 total (MVP + testing) **Pricing Model Preference:** Pay-per-use (scales with actual usage) **Production Cost Targets:**

- 100 users: ~$5/month
- 1,000 users: ~$75/month
- 10,000 users: ~$600/month
- 100,000 users: ~$5,500/month

Key Cost Driver: AI API calls (80-90% of costs at scale)

3. Time to Ship

MVP Deadline: 24 hours (hard gate) **Full Project:** 7 days **Priority:** Speed to market > long-term maintainability (with documentation of technical debt)

Strategy: Leverage Firebase's managed services to minimize infrastructure setup time.

4. Compliance & Regulatory Needs

Requirements: Minimal for MVP

- Basic authentication security
- No HIPAA, GDPR, SOC 2 requirements for initial version
- Data residency: Not specified (Firebase default regions acceptable)

5. Team & Skill Constraints

Team Size: Solo developer **Existing Skills:** React, JavaScript **Learning Appetite:** Moderate (TypeScript adoption acceptable) **Knowledge Gaps:** Real-time sync patterns, WebSocket management, Canvas APIs

Decision: Choose familiar technologies (React) with incremental complexity additions (TypeScript, Konva.js).

Phase 2: Architecture Discovery

6. Database & Real-Time Sync Strategy

Decision: Hybrid Firebase Approach

Firebase Realtime Database - Ephemeral Data:

- Cursor positions (50ms latency requirement)
- User presence (who's online)
- Temporary selections/highlights

Firestore - Persistent Data:

- Board objects (sticky notes, shapes, frames)
- User profiles
- Board metadata and permissions

Why Hybrid? ✅ Realtime DB provides <50ms latency for cursor sync (meets performance target) ✅ Firestore offers better data modeling for complex board objects ✅ Firestore has superior querying capabilities ✅ This is the industry-standard pattern for collaborative applications

Tradeoff Accepted: ❌ Increased complexity managing two databases ❌ Need to coordinate data between systems ❌ More security rules to maintain

Alternatives Considered:

- **Firestore only:** Latency ~100-200ms, too slow for cursor sync
- **Realtime DB only:** Poor data structure, limited querying
- **Third-party (Liveblocks, Partykit):** Better DX but adds vendor dependency and cost

Data Structure:

```
// Realtime Database (cursors)
/boards/{boardId}/presence/{userId}
  - x: number
  - y: number
  - name: string
  - color: string
  - lastSeen: timestamp

// Firestore (board objects)
/boards/{boardId}
  - name: string
```

```
  - createdBy: userId
  - createdAt: timestamp

/boards/{boardId}/objects/{objectId}
  - type: 'sticky' | 'shape' | 'text' | 'frame'
  - x, y, width, height: number
  - content: string
  - color: string
  - userId: string
  - timestamp: number
```

7. Frontend Framework & Canvas Library

Decision: React + TypeScript + Vite + Konva.js

React 18:

- ✅ Familiar technology (existing skill set)
- ✅ Excellent Firebase integration via hooks
- ✅ Large ecosystem and community support
- ✅ Efficient re-rendering for real-time updates

TypeScript:

- ✅ Type safety prevents sync bugs (critical for multiplayer)
- ✅ Autocomplete for Firebase APIs and object structures
- ✅ Self-documenting code for AI tool schemas
- ✅ Catches errors at compile time vs runtime

Why TypeScript matters for this project: Without types, a single typo (`posX` vs `x`) in an object update can break synchronization for all connected users. Type safety is insurance against silent failures in real-time collaboration.

Vite:

- ✅ Fast dev server with HMR
- ✅ Modern bundling (ESM)
- ✅ Excellent TypeScript support out of the box
- ✅ Industry standard (replacing Create React App)

Konva.js (react-konva):

- ✅ React-native API (fits existing skills)
- ✅ Good performance (handles 500+ objects requirement)
- ✅ Built-in object manipulation (drag, resize, rotate)
- ✅ Strong TypeScript definitions
- ✅ Active maintenance and community

Alternatives Considered:

- **Fabric.js:** More mature but heavier bundle, less React-friendly
- **Tldraw:** Purpose-built for whiteboards but too opinionated, reduces learning
- **HTML5 Canvas (vanilla):** Too low-level, would slow MVP development
- **PixiJS:** Better for games, overkill for this use case

Tradeoff Accepted:

- TypeScript learning curve (mitigated by developer's React experience)
- More manual work than Tldraw (acceptable for educational value)

8. AI Integration Architecture

Decision: Anthropic Claude via Firebase Cloud Functions

Architecture Flow:

```
User Input → React App → Firebase Cloud Function → Claude API
                                    ↓
                            Firestore (board updates)
                                    ↓
                            Real-time sync to all users
```

Why Claude (Anthropic)?

- ✅ Superior function calling capabilities
- ✅ Better at structured outputs (critical for layout commands)
- ✅ Longer context windows (8K+ tokens)
- ✅ Excellent documentation for tool use

Why Firebase Cloud Functions?

- ✅ Secure API key management (not exposed to client)
- ✅ Server-side validation and rate limiting
- ✅ Centralized cost tracking and logging
- ✅ Can implement user quotas and abuse prevention
- ✅ Integrated with Firebase Auth for user context

Cost Comparison:

Provider	Input Tokens	Output Tokens	Typical Command Cost
Claude (Anthropic)	$3/M	$15/M	~$0.005
GPT-4 (OpenAI)	$2.50/M	$10/M	~$0.004

Decision: Claude's superior structured outputs justify slight cost premium.

AI Tool Schema (Minimum Viable):

```typescript
interface AITools {
  createStickyNote(text: string, x: number, y: number, color: string): string;
  createShape(type: 'rectangle' | 'circle', x: number, y: number, width: number,
height: number, color: string): string;
  createFrame(title: string, x: number, y: number, width: number, height: number):
string;
  createConnector(fromId: string, toId: string, style: 'arrow' | 'line'): string;
  moveObject(objectId: string, x: number, y: number): void;
  resizeObject(objectId: string, width: number, height: number): void;
  updateText(objectId: string, newText: string): void;
  changeColor(objectId: string, color: string): void;
```

```
  getBoardState(): BoardObject[];
}
```

Alternatives Considered:

- **Client-side Claude API:** Exposes API keys, no rate limiting
- **OpenAI GPT-4:** Faster but weaker structured outputs
- **Open-source models (Llama):** Insufficient function calling quality

Tradeoff Accepted:

- Cloud Function cold starts (1-2s latency, acceptable within <2s AI response budget)
- Vendor lock-in to Anthropic (acceptable for MVP)

9. Authentication & Authorization

Decision: Firebase Authentication

Providers Enabled:

- Google Sign-In (primary, easiest for demo)
- Email/Password (baseline auth)
- Anonymous Auth (optional, for quick testing)

Why Firebase Auth?

- ✅ Built-in integration with Firestore/Realtime DB security rules
- ✅ Automatic token refresh and session management
- ✅ Free tier: 10,000 monthly active users
- ✅ No backend code needed for auth flows
- ✅ Industry-standard security practices

Security Rules Example:

```
// Firestore Rules
rules_version = '2';
service cloud.firestore {
  match /boards/{boardId} {
    allow read: if request.auth != null;
    allow create: if request.auth != null;

    match /objects/{objectId} {
      allow read: if request.auth != null;
      allow create: if request.auth != null;
      allow update, delete: if request.auth != null
        && (resource.data.userId == request.auth.uid ||
get(/databases/$(database)/documents/boards/$(boardId)).data.ownerId ==
request.auth.uid);
    }
  }
}

// Realtime Database Rules
{
  "rules": {
```

```
    "boards": {
      "$boardId": {
        "presence": {
          "$userId": {
            ".write": "$userId === auth.uid",
            ".read": "auth != null"
          }
        }
      }
    }
  }
}
```

Authorization Strategy:

- Board-level permissions (owner, collaborator)
- Object-level ownership (only creator can delete)
- Presence data: users can only write their own cursor position

10. Deployment Strategy

Decision: Vercel (Frontend) + Firebase (Backend/Functions)

Vercel for React App:

- ✅ Generous free tier (100GB bandwidth, unlimited deployments)
- ✅ Automatic deployments from Git (push to deploy)
- ✅ Global CDN (Edge Network)
- ✅ Environment variables management
- ✅ Preview deployments for testing

Firebase for Cloud Functions:

- ✅ Integrated with Firebase services
- ✅ Free tier: 2M invocations/month, 400K GB-seconds
- ✅ Deploy via Firebase CLI
- ✅ Automatic scaling

CI/CD Strategy:

- Git push → Vercel auto-deploys frontend
- Manual Firebase deploy for functions (can automate later)
- Environment-specific Firebase projects (dev, prod)

Alternatives Considered:

- **Firebase Hosting only:** Less flexible for React apps, harder setup
- **Netlify:** Similar to Vercel but less optimized for React/Vite
- **Railway/Render:** Requires containerization, slower for MVP

Phase 3: Post-Stack Refinement

12. Security Vulnerabilities & Mitigation

Known Risks:

1. **Firestore Security Rule Bypass**
    - Risk: Misconfigured rules allow unauthorized data access
    - Mitigation: Test rules with Firebase emulator, use rule simulator
2. **Rate Limiting AI Commands**
    - Risk: User spam creates high API costs
    - Mitigation: Implement per-user quotas in Cloud Functions
3. **XSS via User-Generated Content**
    - Risk: Sticky note text contains malicious scripts
    - Mitigation: Sanitize text inputs, use React's built-in XSS protection
4. **Cursor Position Spam**
    - Risk: Malicious client floods cursor updates
    - Mitigation: Client-side throttling (50ms), server-side rate limits

Security Checklist:

- ✅ Firebase rules tested in emulator
- ✅ API keys in environment variables (never committed)
- ✅ HTTPS only (enforced by Vercel/Firebase)
- ✅ Input validation on Cloud Functions
- ✅ User quotas for AI commands (10-20 per session)

13. File Structure & Project Organization

```
collabboard/
├── src/
│   ├── components/          # React components
│   │   ├── Board/           # Main board container
│   │   ├── Objects/         # Sticky notes, shapes, frames
│   │   ├── Toolbar/         # Object creation tools
│   │   ├── Cursors/         # Multiplayer cursors
│   │   └── AIPanel/         # AI command interface
│   ├── canvas/              # Konva.js canvas logic
│   │   ├── objects/         # Object rendering
│   │   ├── transforms/      # Move, resize, rotate
│   │   └── selection/       # Multi-select logic
│   ├── firebase/            # Firebase configuration
│   │   ├── config.ts        # Firebase init
│   │   ├── firestore.ts     # Firestore hooks
│   │   ├── realtime.ts      # Realtime DB hooks
│   │   └── auth.ts          # Auth hooks
│   ├── ai/                  # AI integration
│   │   ├── commands.ts      # Command parsing
│   │   └── tools.ts         # Tool schema definitions
│   ├── hooks/               # Custom React hooks
│   │   ├── useBoard.ts      # Board state management
│   │   ├── usePresence.ts   # User presence
│   │   └── useSync.ts       # Real-time sync
│   ├── types/               # TypeScript types
```

```
│   │   ├── board.ts         # Board object types
│   │   ├── user.ts          # User types
│   │   └── ai.ts            # AI tool types
│   ├── utils/               # Utility functions
│   ├── App.tsx              # Main app component
│   └── main.tsx             # Entry point
├── functions/               # Firebase Cloud Functions
│   ├── src/
│   │   ├── ai/              # AI command handlers
│   │   └── index.ts         # Function exports
│   └── package.json
├── public/                  # Static assets
├── .env.example             # Environment variables template
├── firebase.json            # Firebase config
├── firestore.rules          # Firestore security rules
├── database.rules.json      # Realtime DB security rules
├── package.json
├── tsconfig.json
├── vite.config.ts
└── README.md
```

14. Naming Conventions & Code Style

TypeScript/React Conventions:

- **Components:** PascalCase (`StickyNote.tsx` , `BoardCanvas.tsx`)
- **Hooks:** camelCase with `use` prefix (`useBoard.ts` , `usePresence.ts`)
- **Types/Interfaces:** PascalCase (`BoardObject` , `User` , `AICommand`)
- **Functions:** camelCase (`createStickyNote` , `updateObjectPosition`)
- **Constants:** UPPER_SNAKE_CASE (`MAX_OBJECTS` , `DEFAULT_COLOR`)

File Naming:

- Components: `ComponentName.tsx`
- Hooks: `useHookName.ts`
- Types: `typeName.ts`
- Utils: `utilName.ts`

Code Style:

- **Formatter:** Prettier (default config)
- **Linter:** ESLint with React + TypeScript rules
- **Import Order:** External → Internal → Types → Styles

15. Testing Strategy

MVP Testing Focus: Manual testing with multiple browser windows

Critical Test Scenarios:

1. ✅ 2 users editing simultaneously in different browsers
2. ✅ One user refreshing mid-edit (state persistence check)
3. ✅ Rapid creation and movement of sticky notes (sync performance)
4. ✅ Network throttling and disconnection recovery (Chrome DevTools)

5. ✅ 5+ concurrent users without degradation

Testing Approach:

- **Unit tests:** Deferred to post-MVP (time constraint)
- **Integration tests:** Manual Firebase emulator testing
- **E2E tests:** Manual browser testing (Chrome, Firefox, Safari)
- **Performance:** Chrome DevTools Performance tab, network throttling

Performance Targets:

- Frame rate: 60 FPS during pan, zoom, object manipulation
- Object sync latency: <100ms
- Cursor sync latency: <50ms
- Object capacity: 500+ objects without performance drops
- Concurrent users: 5+ without degradation

Tools:

- Chrome DevTools (Network tab, Performance tab)
- Firebase Emulator Suite (local testing)
- Lighthouse (performance auditing post-MVP)

16. Recommended Tooling & Developer Experience

VS Code Extensions:

- ESLint
- Prettier
- Firebase Explorer
- TypeScript Vue Plugin (for better TS support)
- GitLens (version control visualization)

CLI Tools:

- Firebase CLI (`npm install -g firebase-tools`)
- Vite dev server (built-in)
- TypeScript compiler (`tsc --watch` for type checking)

Debugging Setup:

- React Developer Tools (browser extension)
- Firebase Emulator UI (local testing)
- VS Code debugger config for Chrome

Development Workflow:

1. Start Firebase emulators: `firebase emulators:start`
2. Start Vite dev server: `npm run dev`
3. Open multiple browser windows for multiplayer testing
4. Use Chrome DevTools network throttling to simulate latency

---

Cost Analysis

Development Costs (7 Days)

Service	Usage	Cost
Firebase (Firestore + Realtime DB)	Testing, <1GB data	$0 (free tier)
Firebase Cloud Functions	~500 invocations	$0 (free tier)
Anthropic Claude API	~1,000 AI commands (testing)	~$10-15
Vercel Hosting	Unlimited deployments	$0 (free tier)
Total		**~$10-15**

Production Cost Projections

Assumptions:

- Average 10 AI commands per user per month
- Average 2,000 tokens per AI command (1,000 input, 1,000 output)
- 5 board sessions per user per month
- Average 50 object updates per session

Users	AI API Costs	Firebase Costs	Total/Month
100	$5 (1K commands)	$0 (free tier)	**$5**
1,000	$50 (10K commands)	$25 (reads/writes)	**$75**
10,000	$500 (100K commands)	$100 (bandwidth + operations)	**$600**
100,000	$5,000 (1M commands)	$500 (scale costs)	**$5,500**

Cost Breakdown at 10,000 Users:

- AI API (Claude): ~83% of costs
- Firestore operations: ~10%
- Firebase Realtime DB: ~5%
- Hosting/bandwidth: ~2%

Cost Optimization Strategies:

1. Implement user quotas (10-20 AI commands per session)
2. Cache common AI responses (e.g., "create SWOT analysis" templates)
3. Use GPT-3.5-turbo for simple commands, Claude for complex ones
4. Batch Firestore writes to reduce operation costs
5. Implement client-side cursor interpolation to reduce Realtime DB writes

---

Risk Assessment & Mitigation

High-Priority Risks

1. Firestore Latency Feels Sluggish

- **Risk:** 100-200ms latency for object creation/movement
- **Impact:** Poor user experience, fails performance targets
- **Mitigation:**

    - Implement optimistic updates (immediate local render)
    - Use Firestore local cache
    - Consider Realtime DB for frequently-updated objects

2. Cloud Function Cold Starts

- **Risk:** 1-2s cold start delays AI responses
- **Impact:** Occasional slow AI commands
- **Mitigation:**
    - Keep functions warm with scheduled pings (costs ~$1/month)
    - Set minimum instances to 1 (costs ~$6/month)
    - Accept cold starts as acceptable within <2s AI budget

3. Cursor Sync Performance at 5+ Users

- **Risk:** Too many cursor updates overwhelm Realtime DB
- **Impact:** Laggy cursors, poor multiplayer experience
- **Mitigation:**
    - Client-side throttling (send updates every 50ms max)
    - Cursor interpolation on receiving clients
    - Test with 10+ users before claiming success

4. Firebase Free Tier Limits During Testing

- **Risk:** Hit rate limits or quotas during development
- **Impact:** Development slowdowns
- **Mitigation:**
    - Use Firebase emulator for local development
    - Monitor usage in Firebase console
    - Upgrade to Blaze (pay-as-you-go) if needed

Medium-Priority Risks

5. Simultaneous AI Commands from Multiple Users

- **Risk:** Race conditions, conflicting board updates
- **Impact:** Objects overlap or wrong placement
- **Mitigation:**
    - Queue AI commands server-side
    - Use Firestore transactions for atomic updates
    - Show "AI is working" indicator to all users

6. TypeScript Learning Curve

- **Risk:** Developer unfamiliar with TypeScript slows development
- **Impact:** Missed 24-hour MVP deadline
- **Mitigation:**
    - Use `any` types where needed to move fast
    - Refine types iteratively
    - Accept some type safety tradeoffs for speed

Key Learnings & Insights

What This Pre-Search Process Revealed

1. **Real-time collaboration is the core complexity** - not the whiteboard features themselves. Separating ephemeral (cursors) from persistent (objects) data is critical.
2. **Type safety is insurance, not overhead** - for multiplayer apps, TypeScript prevents silent failures that break sync for all users.
3. **AI costs will dominate at scale** - 80-90% of production costs will be Claude API calls. User quotas are essential.
4. **Firebase hybrid approach is industry-standard** - Realtime DB for high-frequency updates, Firestore for structured data.
5. **MVP must prioritize sync reliability over features** - a simple board with bulletproof multiplayer beats a feature-rich board with broken collaboration.

Strategic Tradeoffs Accepted

Tradeoff	Benefit	Cost
Firebase vendor lock-in	Rapid MVP development	Migration difficulty later
TypeScript learning curve	Type safety prevents sync bugs	Slower initial development
Hybrid database (Firestore + Realtime DB)	Optimal latency for different data types	Increased complexity
Cloud Function cold starts	Secure API key management	Occasional 1-2s delay
Pay-per-use pricing	No upfront costs	Unpredictable scaling costs

Critical Success Factors

Must Have: ✅ Cursor sync <50ms latency (Realtime DB required) ✅ Object sync <100ms latency (optimistic updates + Firestore) ✅ 5+ concurrent users without degradation ✅ AI commands <2s response time ✅ State persistence across disconnects

Nice to Have:

- Offline support (PWA)
- Undo/redo functionality
- Version history
- Advanced AI commands (natural layout, auto-organize)

Next Steps: Build Strategy

Priority Order (Start → Finish)

1. **Cursor Sync** (4 hours)
    - Set up Firebase Realtime Database
    - Implement cursor position updates

    - Test with 2+ browser windows

2. **Object Sync** (6 hours)
    - Set up Firestore collections
    - Create sticky note component
    - Implement real-time object updates
    - Test simultaneous object creation
3. **Conflict Handling** (4 hours)
    - Implement last-write-wins or operational transforms
    - Handle simultaneous edits to same object
    - Document chosen approach
4. **State Persistence** (3 hours)
    - Ensure board state survives page refresh
    - Handle reconnection gracefully
    - Implement presence cleanup (onDisconnect)
5. **Board Features** (5 hours)
    - Add shapes (rectangle, circle)
    - Implement move, resize, rotate
    - Add selection (single and multi-select)
6. **AI Commands (Basic)** (4 hours)
    - Set up Firebase Cloud Function
    - Implement Claude API integration
    - Support 3-4 simple creation commands
7. **AI Commands (Complex)** (2 hours)
    - Multi-step template generation (SWOT, journey map)
    - Layout algorithms (grid, spacing)

Total: ~28 hours (fits within 7-day timeline with buffer)

Appendix: Effective Prompts Used

Prompt 1: Stack Narrowing

> *"I'd like to use Firebase"*

Why Effective: Immediately narrowed the solution space and allowed deep-dive into Firebase-specific patterns rather than generic cloud comparisons.

Prompt 2: Language Clarification

> *"What language should I use? I'm familiar with React"*

Why Effective: Provided crucial context about existing skills, enabling tailored TypeScript vs JavaScript recommendation with React-specific reasoning.

Prompt 3: Framework vs Language

> *"Should we incorporate a framework, or is TypeScript by itself fine?"*

Why Effective: Revealed misunderstanding about TypeScript's role, clarified that a UI framework (React) is required alongside TypeScript.

Prompt 4: Pre-Research Request

> *"Can you help me do the pre-research for this project?"*

Why Effective: Initiated structured exploration of architecture decisions before coding, following the project's AI-first methodology.

References & Further Reading

Firebase:

- Firebase Realtime Database Best Practices
- Firestore Data Modeling
- Firebase Security Rules Guide

Real-Time Collaboration:

- Figma's Multiplayer Technology
- CRDTs and Operational Transforms

Konva.js:

- Konva.js Documentation
- React Konva Tutorial

AI Function Calling:

- Anthropic Claude Function Calling
- OpenAI Function Calling Guide

Document Version: 1.0 **Last Updated:** February 16, 2026 **Prepared By:** AI-assisted pre-search methodology **Status:** Complete - Ready to begin development